

Validation de formulaire PHP

Ce chapitre et les suivants montrent comment utiliser PHP pour valider les données d'un formulaire.

Validation de formulaire PHP

Pensez SÉCURITÉ lors du traitement des formulaires PHP !

Ces pages vous montreront comment traiter les formulaires PHP dans un souci de sécurité. Une bonne validation des données du formulaire est importante pour protéger votre formulaire contre les pirates et les spammeurs !

Le formulaire HTML sur lequel nous allons travailler dans ces chapitres contient divers champs de saisie : des champs de texte obligatoires et facultatifs, des boutons radio et un bouton d'envoi :

Exemple de validation de formulaire en PHP

* champ obligatoire

Nom: *

E-mail: *

Website:

Commentaire:

Sexe: ☐ Femme ☐ Homme *

Vos Données:

Les règles de validation du formulaire ci-dessus sont les suivantes :

Champ	Règles de validation
Nom	Requis. + Doit contenir uniquement des lettres et des espaces.
E-mail	Requis. + Doit contenir une adresse électronique valide (avec @ et .).
Website	Facultatif. S'il est présent, il doit contenir une URL valide
Commentaire	Facultatif. Champ de saisie multi-lignes (textarea)
Genre	Obligatoire. Vous devez choisir une option

Nous allons d'abord examiner le code HTML brut du formulaire dans les slides suivants :

Champs de texte

Les champs de nom, d'e-mail et de site Web sont des éléments de saisie de texte et le champ de commentaire est une zone de texte. Le code HTML ressemble à ceci :

```
Nom: <input type="text" name="name">  
E-mail: <input type="text" name="email">  
Site Web: <input type="text" name="website">  
Commentaire: <textarea name="comment" rows="5" cols="40"></textarea>
```

Boutons radio

Les champs de genre sont des boutons radio et le code HTML ressemble à ceci :

Sexe:

```
<input type="radio" name="gender" value="female">Femme  
<input type="radio" name="gender" value="male">Homme
```

L'élément de formulaire

Le code HTML du formulaire ressemble à ceci :

```
<form method="post" action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]);?>">
```

Lorsque le formulaire est soumis, les données du formulaire sont envoyées avec méthode="post".

Qu'est-ce que la variable `$_SERVER["PHP_SELF"]` ?

Le `$_SERVER["PHP_SELF"]` est une super variable globale qui renvoie le nom de fichier du script en cours d'exécution.

Ainsi, le `$_SERVER["PHP_SELF"]` envoie les données du formulaire soumis à la page elle-même, au lieu de passer à une autre page. De cette façon, l'utilisateur recevra des messages d'erreur sur la même page que le formulaire.

Qu'est-ce que la fonction `htmlspecialchars()` ?

La fonction `htmlspecialchars()` convertit les caractères spéciaux en entités HTML. Cela signifie qu'elle remplace les caractères HTML tels que `<et>` par `<` ; et `>` ;. Cela empêche les attaquants d'exploiter le code en injectant du code HTML ou Javascript (attaques Cross-site Scripting) dans les formulaires.

Grande note sur la sécurité des formulaires PHP

La variable `$_SERVER["PHP_SELF"]` peut être utilisée par des pirates !

Si `PHP_SELF` est utilisé dans votre page, un utilisateur peut entrer une barre oblique (/) puis quelques commandes Cross Site Scripting (XSS) à exécuter.

Le **Cross Site Scripting (XSS)** est un type de vulnérabilité de sécurité informatique que l'on trouve généralement dans les applications Web. XSS permet aux attaquants d'injecter un script côté client dans les pages Web consultées par d'autres utilisateurs.

Supposons que nous ayons le formulaire suivant dans une page nommée "test_form.php":

```
<form method="post" action="<?php echo $_SERVER["PHP_SELF"];?>">
```

Maintenant, si un utilisateur entre l'URL normale dans la barre d'adresse comme "http://www.example.com/test_form.php", le code ci-dessus sera traduit en :

```
<form method="post" action="test_form.php">
```

Jusqu'ici tout va bien.

Cependant, considérez qu'un utilisateur saisit l'URL suivante dans la barre d'adresse :

```
http://www.example.com/test_form.php/%22%3E%3Cscript%3Ealert('hacked')%3C/script%3E
```

Dans ce cas, le code ci-dessus sera traduit en :

```
<form method="post" action="test_form.php/"><script>alert('hacked')</script>
```

Ce code ajoute une balise de script et une commande d'alerte. Et lors du chargement de la page, le code JavaScript sera exécuté (l'utilisateur verra une boîte d'alerte). Ceci est juste un exemple simple et inoffensif de la façon dont la variable PHP_SELF peut être exploitée.

Sachez que **tout code JavaScript peut être ajouté à l'intérieur de la balise**

<script> ! Un pirate peut rediriger l'utilisateur vers un fichier sur un autre serveur, et ce fichier peut contenir un code malveillant qui peut modifier les variables globales ou soumettre le formulaire à une autre adresse pour enregistrer les données de l'utilisateur, par exemple.

Comment éviter les exploits \$_SERVER["PHP_SELF"] ?

Les exploits \$_SERVER["PHP_SELF"] peuvent être évités en utilisant la fonction htmlspecialchars(). Le code du formulaire devrait ressembler à ceci :

```
<form method="post" action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]);?>">
```

La fonction htmlspecialchars() convertit les caractères spéciaux en entités HTML. Maintenant, si l'utilisateur essaie d'exploiter la variable PHP_SELF, cela se traduira par la sortie suivante :

```
<form method="post" action="test_form.php/&quot;&gt;&lt;script&gt;alert('hacked')&lt;/script&gt;">
```

La tentative d'exploit échoue et aucun mal n'est fait !

Valider les données de formulaire avec PHP

La première chose que nous allons faire est de passer toutes les variables via la fonction `htmlspecialchars()` de PHP. Lorsque nous utilisons la fonction `htmlspecialchars()` ; puis si un utilisateur essaie de soumettre ce qui suit dans un champ de texte :

```
<script>location.href('http://www.hacked.com')</script>
```

- cela ne serait pas exécuté, car il serait enregistré sous forme de code échappé HTML, comme ceci :

```
<script>location.href('http://www.hacked.com')</script>
```

Le code peut maintenant être affiché en toute sécurité sur une page ou dans un e-mail.

Nous ferons également deux autres choses lorsque l'utilisateur soumettra le formulaire :

1. Supprimez les caractères inutiles (espace supplémentaire, tabulation, saut de ligne) des données d'entrée de l'utilisateur (avec la fonction **PHP trim()**)
2. Supprimez les barres obliques inverses (\) des données d'entrée de l'utilisateur (avec la fonction **PHP stripslashes()**)

L'étape suivante consiste à créer une fonction qui effectuera toutes les vérifications pour nous (ce qui est beaucoup plus pratique que d'écrire le même code encore et encore).

Nous nommerons la fonction **test_input()**.

Maintenant, nous pouvons vérifier chaque variable \$_POST avec la fonction test_input(), et le script ressemble à ceci :

```
<?php
// définir des variables et les mettre à des valeurs vides
$name = $email = $gender = $comment = $website = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = test_input($_POST["name"]);
    $email = test_input($_POST["email"]);
    $website = test_input($_POST["website"]);
    $comment = test_input($_POST["comment"]);
    $gender = test_input($_POST["gender"]);
}

function test_input($data) {
    $data = trim($data);
    $data = stripslashes($data);
    $data = htmlspecialchars($data);
    return $data;
}
?>
```

Notez qu'au début du script, nous vérifions si le formulaire a été soumis à l'aide de `$_SERVER["REQUEST_METHOD"]`. Si `REQUEST_METHOD` est `POST`, alors le formulaire a été soumis - et il doit être validé. S'il n'a pas été soumis, ignorez la validation et affichez un formulaire vierge. Cependant, dans l'exemple ci-dessus, tous les champs de saisie sont facultatifs. Le script fonctionne bien même si l'utilisateur n'entre aucune donnée. L'étape suivante consiste à rendre les champs de saisie obligatoires et à créer des messages d'erreur si nécessaire.